CompressGrad: Recovering the Accuracy Lost to Prompt Compression with Question-Aware Selection and TextGrad-Guided Repair

Krrish Singla *PES1UG24AM141*
Kishan K Shenoy *PES1UG24AM138*
Leeladhar G Sairaju *PES1UG24AM148*
Mohammad Bilal A *PES1UG24AM162*

Mini Project, Prompt Engineering and Optimization for Generative AI (UE24AM343AB5)
B.Tech Computer Science & Engineering (AI & ML), Semester 5, PES University, Bengaluru, India

***Abstract*—Prompt compression methods such as LLMLingua reduce the token cost of long-context LLM pipelines, but aggressive compression degrades answer quality. On 100 HotpotQA questions, compressing the context by 2×, 5×, 10× and 20× with LLMLingua-2 lowers F1 from 0.740 (uncompressed, 1,452 prompt tokens on average) to 0.676, 0.483, 0.410 and 0.333. We test two ways to recover this loss. The first is a TextGrad repair loop in which a judge model that sees only the compressed context, never the gold answer, critiques the draft answer and rewrites it. After error analysis exposed two failure modes, a revised loop raises F1 by +0.105 at 5× and +0.114 at 10× over 90 questions, but replaying every stored call shows that its judge calls use more prompt tokens than sending the uncompressed context. The second is question-aware compression: an off-the-shelf reranker and a bridge-entity rule choose the paragraphs before LLMLingua-2 compresses them, under the same per-question token budget. The gold answer then survives compression far more often (46% to 86% at 5×), and F1 rises to 0.727, 0.691, 0.681 and 0.555, with exact-match gains that are significant at 5×, 10× and 20× (McNemar $p \leq 0.0005$). At 10× it keeps 92% of the uncompressed F1 with 17% of the prompt tokens. On question-aware contexts, repair adds no significant further gain. These are pilot results from one dataset and one model pair.**

***Index Terms*—prompt compression, LLMLingua, TextGrad, question answering, reranking, LLM-as-judge, prompt optimization**

I. Introduction

Large language models are increasingly used with long inputs such as documents, retrieved passages and enterprise records, and both API cost and latency grow with the number of prompt tokens. Prompt compression methods such as LLMLingua [1], LongLLMLingua [2] and LLMLingua-2 [3] shorten a prompt by removing tokens that a small model judges to carry little information, and they report large savings with limited loss on their benchmarks.

The accuracy side of this trade-off is less well mapped. Compression removes words the downstream model may need, and published results mostly describe compression on its own. Two questions remain open: at what compression ratio does answer quality fall off, and can part of the lost accuracy be recovered without sending the full prompt again?

TextGrad [4] offers one way to try. An LLM critiques a variable (here, a draft answer) in natural language, and this critique, treated as a "textual gradient," is used to revise the variable. The survey we carried out for Review 1 found no work that combines TextGrad with LLMLingua-style compression, although we cannot rule out that such work exists. A second, cheaper way is to make the compressor itself look at the question, so that the words the answer needs are less likely to be removed.

We study five research questions. **RQ1:** How much accuracy does LLMLingua-2 compression cost at 2×, 5×, 10× and 20×? **RQ2:** Can TextGrad-guided repair, with a judge that never sees the gold answer, recover part of the lost accuracy? **RQ3:** At which compression ratio does repair stop helping? **RQ4:** Does question-aware selection before compression recover more, at the same token budget? **RQ5:** What does repair cost in tokens?

This paper makes six contributions:

- We measure the compression–accuracy curve of LLMLingua-2 on 100 HotpotQA questions at 2× to 20×.
- We build a compression-then-repair pipeline in which a judge that sees only the compressed context guides TextGrad to revise a draft answer, and explain through error analysis why a first version failed.
- We propose a simple question-aware front end (a reranker plus a bridge-entity rule) that keeps the same per-question token budget as LLMLingua-2 and test it with paired significance tests.
- We evaluate repair on question sets not used for design, on both kinds of compressed context.
- We measure the token cost of repair by replaying every stored model call.
- We report negative results: a stricter answer prompt did not help, and repair does not save tokens.

II. Related Work

A. Prompt Compression

LLMLingua [1] uses a small language model to identify and drop unimportant tokens. The compressed prompt is often hard for people to read but remains usable by the target LLM, and

the paper reports compression of up to 20× with small loss on several benchmarks. LongLLMLingua [2] makes compression question-aware: a coarse step ranks documents by their relevance to the question and a fine step compresses tokens, and it reorders documents against the "lost in the middle" bias [9]. LLMLingua-2 [3] treats compression as token classification with a small encoder trained on data distilled from GPT-4, and is reported to be several times faster than LLMLingua. Selective Context [5] prunes tokens by self-information. LLMLingua-2 and Selective Context do not look at the question. Our question-aware front end follows the idea of LongLLMLingua's coarse step, but uses an off-the-shelf cross-encoder reranker [12] and keeps LLMLingua-2 for the token step.

B. Textual Gradients

TextGrad [4] treats natural-language feedback from an LLM as a gradient and passes it backwards through a computation graph, with an interface modelled on PyTorch. In TextGrad a Variable holds a piece of text together with a short description of its role. A TextLoss wraps an evaluation instruction: given a variable, an LLM writes a critique of it. Calling backward() asks an LLM to turn that critique into feedback that is stored as the variable's gradient, and the Textual Gradient Descent (TGD) optimizer prompts an LLM with the variable, its gradient and optional constraints to produce an improved value. In our pipeline the only variable is the draft answer.

C. LLM-as-Judge and Iterative Refinement

Zheng et al. [6] show that strong LLM judges often agree with human raters but have known biases. In our system the judge produces feedback rather than the final score: final scoring uses exact match and F1 against the gold answers. Self-Refine [10] and Reflexion [11] revise outputs with verbal feedback; our repair loop is a small instance of this family in which the draft comes from a compressed prompt and the judge sees only the compressed text.

D. Positioning

Table I summarizes how this work relates to the methods above.

TABLE I
POSITIONING OF THIS WORK

Work	Compresses prompts	Question-aware	Repairs with text feedback
LLMLingua, LLMLingua-2	✓	–	–
LongLLMLingua	✓	✓	–
TextGrad, Self-Refine	–	–	✓
CompressGrad (this work)	✓	✓	✓

III. METHOD

A. Pipeline Overview

Given a context C and a question q, a compressor returns a shorter context C_r with roughly |C|/r tokens. The answer model produces a draft answer a_0 from (C_r, q). An optional repair step produces a revised answer a* from (C_r, q, a_0). Only the context is compressed; the instruction and the question are left intact. Fig. 1 shows the pipeline. Nothing from the gold answers is available to the compressor, the answer model or the judge, so every step could be applied to new questions in the same way.

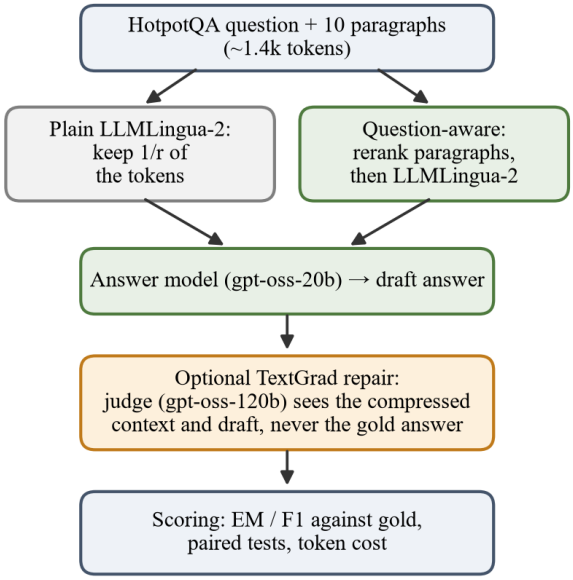


Fig. 1. The CompressGrad pipeline. Both compressors produce the same number of tokens per question. The judge never receives the gold answer.

B. Plain Compression

We use LLMLingua-2 through the llmlingua Python package with the llmlingua-2-xlm-roberta-large-meetingbank model. It labels each token as keep or drop with a bidirectional encoder and keeps the tokens with the highest keep probability until the budget is reached, preserving word order. For a ratio r the keep rate is 1/r; newlines are forced to be kept and consecutive duplicates are dropped. LLMLingua-2 sees the context only, so it spends the budget on the eight distractor paragraphs of a HotpotQA question as much as on the two that matter.

C. Question-Aware Compression

Our front end works on the ten "Title: text" paragraphs of a context (Algorithm 1). A cross-encoder reranker, BAAI/bge-reranker-base [12], scores each paragraph against the question. HotpotQA questions need two paragraphs, and the second (the bridge) often shares few words with the question, so a bridge rule moves up any paragraph whose title is mentioned in the best paragraph, or that mentions the best paragraph's title. Paragraphs are then taken in rank order until their length reaches the budget, and LLMLingua-2 trims them to exactly the number of tokens that plain compression used for the same question, so the two compressors are compared at equal cost. We tried keeping paragraphs worth 1, 1.5, 2 and 3

times the budget before trimming; 1 was best for every ratio, which means it is better to keep fewer paragraphs and cut them less. A sentence-level variant picks the best sentences from the top four paragraphs instead.

Algorithm 1: Question-aware compression of one context

```
Input: paragraphs p1..p10, question q, budget T
 1: s_i <- reranker(q, p_i)        for each i
 2: t <- argmax_i s_i
 3: for i != t:
 4:   if title(p_i) in p_t or title(p_t) in p_i:
 5:     s_i <- s_i + 3              # bridge hop
 6: K <- [] ; take p_i by decreasing s_i
 7:   until tokens(K) >= T and |K| >= 2
 8: if tokens(K) > T: K <- LLMLingua2(K, T)
 9: return K
```

D. TextGrad Repair Loop

The draft answer is a TextGrad Variable with gradients enabled. A TextLoss is built from a fixed evaluation instruction that contains the question and the compressed context; it asks the judge whether the draft is supported by the context, answers exactly what the question asks, and is short. backward() makes the judge write feedback on the variable, and the TGD step rewrites the answer using that feedback. The judge and optimizer share one model and never see the gold answer or the original context. We evaluate three versions (Algorithm 2).

v1 uses the evaluation instruction above and one repair round. **v2** adds three rules: (i) never answer "unknown" or "not stated"; (ii) if the draft is correct, reply "The draft answer is correct.", which stops the loop and keeps the draft; (iii) keep the short answer form. A guard also keeps the draft if the final answer still abstains; it never fired. **v3** adds to v2 that the judge may only change the draft when the context clearly supports another answer, and that a new answer must use words from the context, and it runs two rounds.

Algorithm 2: Repair of one answer (v2)

```
Input: question q, compressed context Cr,
       draft a0, judge/optimizer model J
 1: a <- Variable(a0, requires_grad=True)
 2: L <- TextLoss(instruction(q, Cr), J)
 3: for round = 1 .. R:
 4:   fb <- L(a)          # judge reads a
 5:   if fb says draft is correct: break
 6:   fb.backward()       # textual gradient
 7:   TGD.step()          # J rewrites a
 8: if a contains an abstention phrase:
 9:   a <- a0             # guard
10: return a
```

E. Metrics and Statistics

We use exact match (EM) and token-level F1 after the standard normalization (lower case, no punctuation, no articles, collapsed spaces). Before scoring, a gold-blind cleanup removes an "Answer:" prefix, text after the first line, bracketed asides such as "(RAF)" and stray periods; applied to the stored predictions it changed the uncompressed EM from 0.53 to 0.56 and never lowered a score at 2×, 10× or 20×. All comparisons in this paper use the same scoring on both sides. The share of lost accuracy that repair wins back is

$$\text{Recovery} = (F1_{\text{rep.}} - F1_{\text{comp.}}) / (F1_{\text{full}} - F1_{\text{comp.}}) \quad (1)$$

computed on the same questions, and only reported when at least 0.05 F1 was lost. To compare two conditions on the same questions we count the questions each one wins and use an exact two-sided sign test; on exact match this is McNemar's exact test [13]. For repair we also report a 95% paired bootstrap range of the mean F1 change (5,000 resamples). As a cheap check that needs no model calls, *answer recall* is the share of non yes/no questions whose normalized gold answer still appears in the compressed context.

F. Middleware and Demo

pipeline.py provides compress_and_answer(context, question, ratio) and compress_and_repair(...), which compress the context with the question-aware front end, answer from it, optionally apply repair v2, and return the compressed context, the token counts, the draft and the final answer. demo.py prints the answers from the full context, from plain LLMLingua-2 and from question-aware compression side by side, with F1 when a gold answer is known; on sample questions it runs from the cache without API calls.

IV. Experimental Setup

A. Data

We use the distractor setting of HotpotQA [7], a multi-hop question answering dataset in which each question comes with ten paragraphs, two of which contain the supporting facts. We sampled 100 questions at random (seed 42) from the 7,405-question validation split. With the instruction and question, an uncompressed prompt has 1,452 tokens on average. For example, the first question asks in what year the university where Sergei Aleksandrovich Tokarev was a professor was founded (gold answer: 1755); it needs two steps, finding the university and then its founding year.

B. Models and Infrastructure

The answer model is gpt-oss-20b and the judge and optimizer is gpt-oss-120b [8], both served through the free tier of the Groq API at temperature 0 with low reasoning effort. The reranker and LLMLingua-2 run locally on the CPU. Every model call is cached, so a rerun gives the same results; the free tier allows 200,000 tokens per day for each model, which is why the repair experiments use sets of 30 questions.

C. Evaluation Protocol

The compression curves for both compressors use all 100 questions. Repair is evaluated on three disjoint sets of 30 questions: questions 1–30 were used to find the failure modes of v1 and design v2, questions 31–60 are held out for v2, and questions 61–90 were not used until v3 had been designed (Fig. 2). Repair on question-aware contexts was run once, with v3 at 5×, 10× and 20× on questions 61–90. Table II lists the settings.

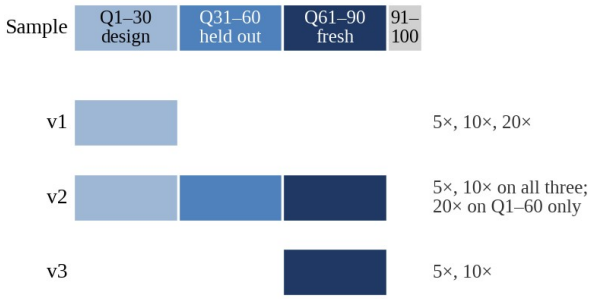


Fig. 2. Question sets and the repair versions evaluated on each on plain contexts. Questions 91–100 were not used for repair.

TABLE II
EXPERIMENTAL SETTINGS

Item	Setting
Dataset	HotpotQA distractor setting, validation split
Sample	100 questions, random seed 42
Compressor	LLMLingua-2, XLM-RoBERTa-large (MeetingBank)
Question-aware	bge-reranker-base + bridge rule, then LLMLingua-2
Levels	2×, 5×, 10×, 20× (keep rate 1/r)
Answer model	gpt-oss-20b via Groq
Judge / optimizer	gpt-oss-120b via Groq
Decoding	Temperature 0, low reasoning effort
Repair	TextGrad TextLoss and TGD; 1 round (v1, v2), 2 rounds (v3)
Statistics	Sign / McNemar exact tests; 5,000-resample bootstrap

D. Engineering Challenges

Model retirement. Our first choice of model, llama-3.3-70b-versatile, was retired on 16 August 2026, so we moved to the gpt-oss models. *Daily token limits.* An uncompressed run over 100 questions needs about 145,000 prompt tokens, 73% of one model's daily allowance, so we added detection of the daily limit, a clean stop and resume from the cache. *Empty replies.* gpt-oss can spend the whole output budget on hidden reasoning and return no text; seven such empty replies had been cached and scored as wrong answers. The caller now retries them with a larger budget and never caches an empty reply. *No GPU.* The compressor and reranker run on the CPU. *Library integration.* TextGrad's engine names did not work with our provider string, so a small engine class routes TextGrad's calls through our cached caller.

E. Implementation

The pipeline is a set of Python scripts (Appendix B): data preparation, the two compressors, evaluation, repair, statistics, cost replay and figures. Scoring and statistics live in a module without API dependencies, covered by unit tests. Repair results are written one question at a time, so an interrupted run resumes where it stopped. The experiments were run on Windows laptops in October and November 2026 with llmlingua 0.2.2, textgrad 0.1.8, litellm 1.106 and sentence-transformers 6.1.

V. RESULTS

A. Compression Alone

Table III shows the compression–accuracy curve of plain LLMLingua-2. Going from the full context to 2× costs 0.064 F1 while removing 47% of the prompt tokens. Beyond that F1 falls quickly: 0.483 at 5×, 0.410 at 10× and 0.333 at 20×. The reason is visible in the compressed text: the gold answer is still present in 82% of the contexts at 2× but in only 46%, 32% and 9% at 5×, 10× and 20× (Table IV), and the answer model almost never answers correctly once it is gone.

TABLE III
PLAIN LLMLINGUA-2 COMPRESSION (n = 100)

Level	Prompt tokens	Token reduction	EM	F1	F1 kept
None	1,452	–	0.560	0.740	100%
2×	765	47%	0.510	0.676	91%
5×	383	74%	0.350	0.483	65%
10×	251	83%	0.320	0.410	55%
20×	182	87%	0.240	0.333	45%

Prompt tokens are averages counted by the answer model and include the instruction and question. Without the answer cleanup the EM/F1 values are 0.530/0.731, 0.490/0.670, 0.340/0.483, 0.320/0.407 and 0.230/0.330, as in our Review 2 report.

B. Question-Aware Compression

Question-aware selection changes what survives. At the same token budget, the gold answer is kept in 98%, 86%, 77% and 48% of the contexts at 2×, 5×, 10× and 20× (Table IV, Fig. 3).

TABLE IV
ANSWER RECALL: GOLD ANSWER STILL IN THE CONTEXT

Level	Plain	Question-aware (paragraphs)	Question-aware (sentences)	Avg tokens
2×	82%	98%	–	~655
5×	46%	86%	91%	~268
10×	32%	77%	79%	~133
20×	9%	48%	56%	~62

96 questions whose answer is not yes/no. Tokens counted with the compressor's tokenizer (context only). The sentence variant is used from 5× up.

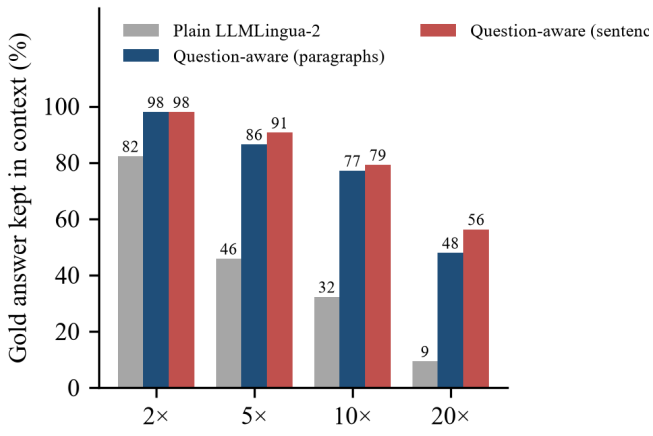


Fig. 3. Share of questions whose gold answer survives compression, at equal token budgets.

Table V gives the effect on accuracy, with the same prompt, the same answer model and the same scoring on both sides, so only the compression differs. Exact match rises by 16, 21 and 17 points at 5×, 10× and 20×, and for every question that question-aware compression breaks it fixes between 7 and 12. These differences are significant (McNemar $p \leq 0.0005$); at 2× the gain is smaller and could be chance ($p = 0.23$). Fig. 4 and Fig. 5 show the curves: at 10×, F1 is 0.681, which is 92% of the uncompressed 0.740, with 249 instead of 1,452 prompt tokens.

TABLE V
PLAIN VS QUESTION-AWARE COMPRESSION (n = 100, PAIRED)

Level	EM plain	EM QA	F1 plain	F1 QA	EM fixed / broken	McNemar p
2×	0.51	**0.56**	0.676	**0.727**	8 / 3	0.23
5×	0.35	**0.51**	0.483	**0.691**	18 / 2	0.0004
10×	0.32	**0.53**	0.410	**0.681**	23 / 2	<0.0001
20×	0.24	**0.41**	0.333	**0.555**	20 / 3	0.0005

QA = question-aware. Average prompt tokens for QA: 758, 381, 249 and 181. The F1 sign test gives p = 0.05, 0.0001, <0.0001 and <0.0001.

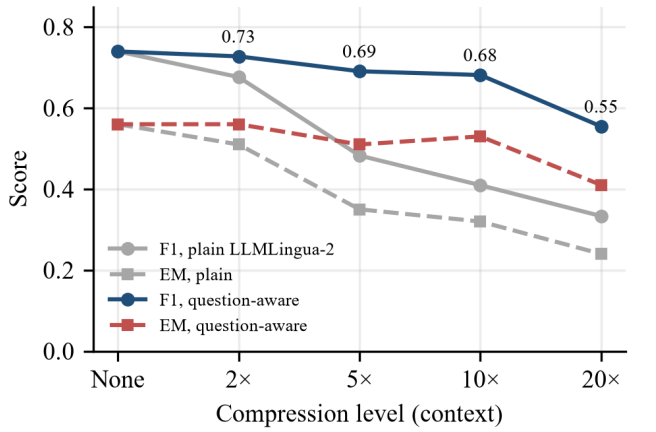


Fig. 4. EM and F1 against compression level for plain LLMLingua-2 (grey) and question-aware compression (colour), 100 questions.

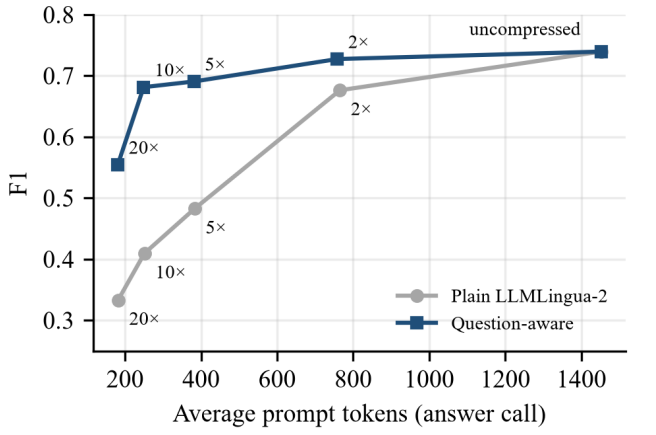


Fig. 5. Token cost against F1. Question-aware compression moves the whole curve up; its 10× point is close to plain 2× in F1 with a third of the tokens.

The sentence variant keeps the answer more often still (91%, 79% and 56% at 5×, 10× and 20×), but its accuracy run did not finish within the free tier's daily limit, so we report it as answer recall only and use the paragraph variant everywhere else.

C. Repair v1 Did Not Help

On questions 1–30, v1 changed F1 by +0.033, +0.027 and −0.050 at 5×, 10× and 20×. It helped and hurt 5 and 5, 6 and 6, and 7 and 8 questions, and the 95% ranges all include zero.

D. Repair v2

Table VI and Fig. 6 pool v2 over the three question sets on plain contexts. It raises F1 by +0.105 at 5× and +0.114 at 10×, and the 95% ranges exclude zero; exact match rises from 0.344 to 0.456 and from 0.333 to 0.433. On exact match, 12 answers were fixed and 2 broken at 5× (McNemar $p = 0.013$) and 12 and 3 at 10× ($p = 0.035$). At 20× the gain is +0.027 and cannot be told apart from zero. On the two sets not used for design, the gain was +0.139 and +0.108 at 5× and +0.129 and +0.078 at 10×.

TABLE VI
REPAIR V2 ON PLAIN CONTEXTS, POOLED

Level	n	Helped / hurt / same	ΔF1 [95% range]	Recovered
5×	90	16 / 5 / 69	+0.105 [+0.026, +0.187]	≈41%
10×	90	18 / 5 / 67	+0.114 [+0.032, +0.199]	≈37%
20×	60	7 / 3 / 50	+0.027 [−0.047, +0.102]	≈7%

ΔF1 is the mean per-question change from the compressed-only draft to the repaired answer. These runs predate the answer cleanup and are scored without it on both sides.

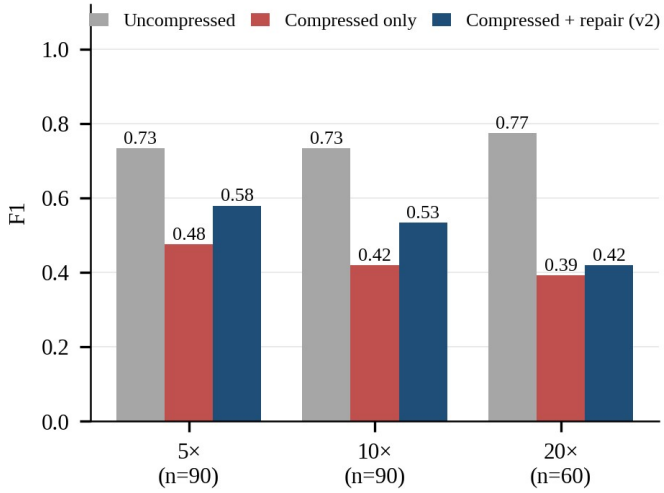


Fig. 6. Uncompressed, compressed-only and repaired (v2) F1 on the pooled question sets of plain contexts. Each level uses its own questions, so the uncompressed bars differ.

E. Comparing the Versions

Table VII compares the versions. On questions 61–90, v3 scored higher than v2 at 5× and lower at 10×, with neither difference clear on 30 questions. v2 hurt one question or none at each level on questions 1–30, while v1 hurt as many as it helped (Fig. 7). We treat v2 as the final repair method and v3 as an ablation.

TABLE VII
MEAN F1 CHANGE FROM REPAIR, BY VERSION (plain CONTEXTS)

Version	Questions	5×	10×	20×
v1	1–30	+0.033	+0.027	−0.050
v2	1–30	+0.067	+0.136	+0.060
v2	31–60	+0.139	+0.129	−0.007
v2	61–90	+0.108	+0.078	–
v3	61–90	+0.133	+0.033	–

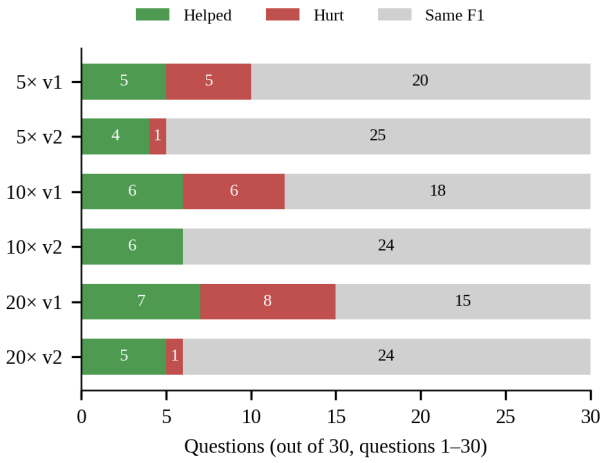


Fig. 7. Questions helped, hurt and unchanged by repair on questions 1–30, for v1 and v2.

F. Repair on Question-Aware Contexts

With better contexts there is less to repair. Table VIII runs v3 on question-aware contexts for questions 61–90, the same questions as the plain v3 run. The question-aware draft alone already beats the plain draft plus repair at 5× (0.658 against 0.523) and 10× (0.727 against 0.457). Repair then adds +0.052 at 5×, −0.005 at 10× and +0.076 at 20×; it helped 3, 1 and 4 questions and hurt 2, 2 and 1, and none of these changes is significant (bootstrap ranges include zero; McNemar p = 1.0, 1.0, 0.63).

TABLE VIII
F1 ON QUESTIONS 61–90: PLAIN VS QUESTION-AWARE, WITH AND WITHOUT REPAIR (v3)

Level	Plain	Plain + repair	QA	QA + repair	ΔF1 [95% range], QA
5×	0.390	0.523	0.658	**0.709**	+0.052 [−0.067, +0.178]
10×	0.424	0.457	**0.727**	0.722	−0.005 [−0.038, +0.029]
20×	–	–	0.541	**0.616**	+0.076 [−0.047, +0.207]

Uncompressed F1 on these questions is 0.672. Plain runs are scored without the cleanup, QA runs with it; within each pair (with and without repair) the scoring is the same.

G. The Cost of Repair

Repair re-sends the compressed context to a larger model. We replayed every stored repair from the cache (cost_analysis.py), which reproduced every stored answer exactly, and added up the prompt tokens of all judge, gradient and optimizer calls (Table IX, Fig. 8). Per question, the answer call plus repair uses 1,550 to 3,670 prompt tokens, 107% to 252% of the 1,452 tokens of the uncompressed prompt, and most of them go to the 120B judge. In our setup repair therefore does not save tokens; question-aware compression recovers more accuracy with no extra model calls.

TABLE IX
PROMPT TOKENS PER QUESTION INCLUDING REPAIR

Run	Level	Answer call	Repair calls	Calls	Total vs full
v2, plain	5×	385	1,762	1.9	148%
v2, plain	10×	251	1,630	2.0	130%
v2, plain	20×	181	1,402	1.9	109%
v3, QA	5×	390	3,276	3.3	252%
v3, QA	10×	253	1,297	1.8	107%
v3, QA	20×	184	1,423	2.0	111%

Averages over the replayable questions (v2: n-weighted over the three sets; 4 of 240 v2 questions had a call missing from the cache). Output tokens are not included.

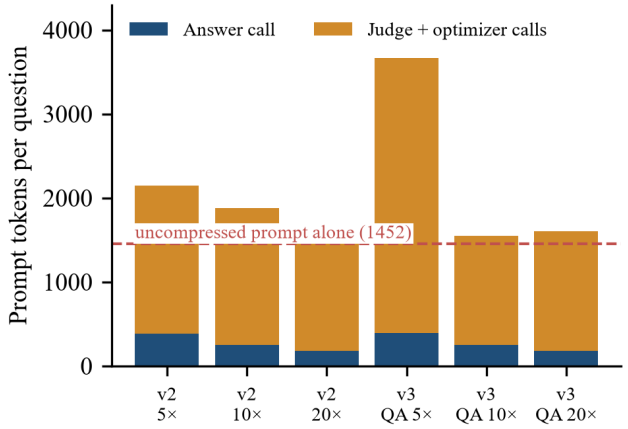


Fig. 8. Prompt tokens per question for the answer call and the repair calls. Every repair configuration exceeds the uncompressed prompt (dashed).

H. Error Analysis

We found three failure modes. *Abstention.* With v1, the judge turned drafts into answers such as “Not stated” and “Insufficient data”: none of the drafts abstained, but 2, 3 and 8 repaired answers did at 5×, 10× and 20×. The v2 rules removed this completely. *Editing correct answers.* The loop also edits answers that are already right: “Gatwick Airport” became “London Gatwick” and “Kevin Spacey” became “Kevin Spacey (actor)”. With the “draft is correct” rule, v2 left about three quarters of the answers unchanged. *Entity swaps.* Some errors remain: “Rudolf Höss” became “Kurt Gerstein” and “Cold War” became “World War I”. When compression removes the sentence that links two entities, the judge can pick a wrong but plausible entity from the remaining text. Question-aware compression attacks this cause directly, because it keeps the linking paragraph more often.

TABLE X
EXAMPLES OF REPAIRED ANSWERS

Case	Gold	Draft	Repaired
v2, 5×, helped	Punjabi/Pashtun	Norwegian	Punjabi/Pashtun
v2, 5×, helped	Puli Alam	Logar	Pul-i-Alam
v1, 10×, gave up	Greyia	Greyia	cannot be determined
v1, 20×, over-edit	Kevin Spacey	Kevin Spacey	Kevin Spacey (actor)
v2, 5×, swap	the Cold War (1947–91)	Cold War	World War I
v3, 5×, swap	Rudolf Höss	Rudolf Höss	Kurt Gerstein
v2, 10×, helped	dinosaur	Red and white "S" logo.	dinosaur

Taken from the result files; illustrative, not a random sample.

I. Consistency Across Question Sets

Fig. 9 shows the share of lost F1 that v2 recovered on each question set. At 5× and 10× it stays between 31.9% and 46.0% on all three sets, including the two not used for design. At 20× it is 16.1% and −1.7%, which is why we do not claim a gain there.

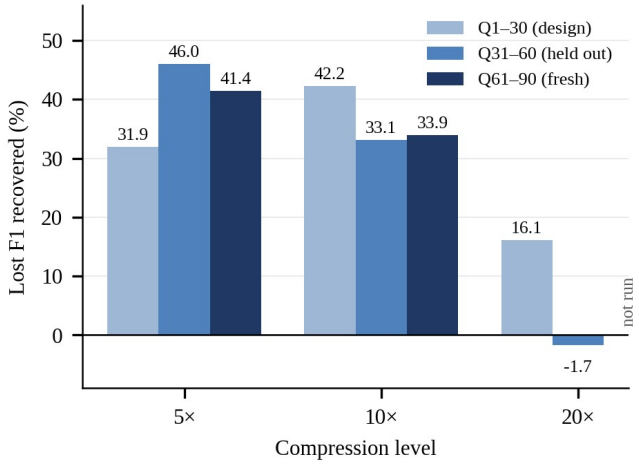


Fig. 9. Share of the lost F1 recovered by repair v2 on each set of 30 questions, plain contexts.

J. Negative Results

A stricter answer prompt that asked the model to copy the exact span from the context did not help: on 93 uncompressed questions it fixed 4 exact-match answers and broke 5 (EM 0.570 against 0.559), so the original prompt is used throughout. Repair, as shown above, costs more tokens than it saves.

K. Summary of Results

- Plain LLMLingua-2 keeps 91% of the F1 at 2×, but only 65%, 55% and 45% at 5×, 10× and 20×, because the gold answer is often removed.
- Question-aware compression at the same budget raises F1 to 0.691, 0.681 and 0.555 at 5×, 10× and 20× (EM +16 to +21 points, McNemar $p \leq 0.0005$).
- Repair v2 raises F1 by about +0.11 at 5× and 10× on plain contexts, but costs 109–148% of the uncompressed prompt in tokens.
- On question-aware contexts repair adds no significant gain.

VI. DISCUSSION AND LIMITATIONS

Why question-aware selection works. HotpotQA contexts are mostly distractors: two of ten paragraphs hold the answer. A question-agnostic compressor spreads its budget evenly and deletes the answer, while a reranker can drop whole irrelevant paragraphs and leave the relevant ones nearly intact. On contexts with fewer distractors the gain may be smaller.

Sample size. The compression curves use 100 questions and each repair set 30. The question-aware gains at 5× to 20× are large enough to be significant at this size; the repair results on question-aware contexts are not, and a few points either way would need several hundred questions.

Tuning on the evaluation sample. The number of paragraphs kept before trimming was chosen on answer recall over the same 100 questions. Only four values were tried and all of them beat plain compression by a wide margin, so the gain does not depend on this choice, but the exact numbers may be slightly optimistic.

Source of the repair gain. The judge is a larger model than the answer model and may use its own knowledge, so part of the repair gain may come from a second opinion by a stronger model. We did not run the control in which the larger model answers directly from the compressed context.

Data contamination and repeatability. HotpotQA is public and the gpt-oss models may have seen it. Hosted models can change; the cache makes the reported numbers repeatable from the stored responses, but a fresh run with new model versions may differ. Output tokens and latency were not measured.

Practical guidance. For accuracy-critical use, question-aware compression at 2× keeps 98% of the uncompressed F1 (0.727 against 0.740) with half the tokens. For cost-critical use, question-aware compression at 10× keeps 92% with 17% of the tokens. Repair is worth its cost only when the answer must come from a context that cannot be selected better, and when calls to a larger model are acceptable.

VII. COMPARISON WITH THE REVIEW 1 PLAN

TABLE XI
REVIEW 1 PLAN AND WHAT WAS DONE

Planned	Done	Status
Literature review, setup	Survey of compression, TextGrad and LLM-as-judge; environment	Done
Dataset and baseline	100 HotpotQA questions, not enterprise documents	Changed
LongLLMLingua and LLMLingua-2	LLMLingua-2, plus our own question-aware front end	Done
Compression tiers, repair loop	2× to 20×; TextGrad repair v1 to v3	Done
QA and summarization	Question answering only	Partly
LLM-judge scoring, latency, cost	EM, F1, significance tests; prompt-token cost measured; no latency	Partly
Middleware, demo, report	pipeline.py, demo.py, this paper	Done

VIII. Conclusion

Compressing HotpotQA contexts with LLMLingua-2 costs little at 2× but F1 falls to 0.483, 0.410 and 0.333 at 5×, 10× and 20×, mainly because the answer itself is deleted. A TextGrad repair loop whose judge sees only the compressed context recovers about 41% and 37% of the lost F1 at 5× and 10× once it is told to avoid "unknown" answers and leave correct drafts alone, but its calls cost more tokens than the full prompt. Choosing what to keep with the question in mind is both cheaper and more effective: at the same budget it raises F1 to 0.691, 0.681 and 0.555.

In terms of the research questions: **RQ1**, compression costs 0.064 F1 at 2× and 0.407 at 20×. **RQ2**, repair v2 recovers about 41% at 5× and 37% at 10×. **RQ3**, repair stops helping reliably between 10× and 20×. **RQ4**, question-aware selection recovers far more, with significant exact-match gains at 5×, 10× and 20×. **RQ5**, repair uses 107–252% of the uncompressed prompt tokens.

Future work: confirm the sentence-level variant on accuracy; scale to more questions and datasets, including summarization; run the control in which the judge model answers directly; measure output tokens and latency; and let repair restore a few tokens from the original context within a fixed budget.

IX. Team And Contributions

The project was carried out by a team of four students. Table XII lists the part each member presented in Review 1.

TABLE XII
Team Members

Member	SRN	Review 1 section
Krrish Singla	PES1UG24AM141	Problem, motivation and research gap
Kishan K Shenoy	PES1UG24AM138	Literature survey: prompt compression
Leeladhar G Sairaju	PES1UG24AM148	Literature survey: optimization, evaluation and the gap
Mohammad Bilal A	PES1UG24AM162	Objective, technical approach and expected outcome

Code, data sample and results:
https://github.com/krrishsingla11/compressgrad

Acknowledgment

The authors thank the faculty of the course Prompt Engineering and Optimization for Generative AI (UE24AM343AB5) at PES University for their guidance during the reviews.

Appendix A: Prompts

The answer prompt, with the (compressed) context and the question inserted, was:

```
Answer the question using the context below.
Reply with ONLY the short answer (a few words),
no explanation.

Context:
[context]
Question: [question]
Answer:
```

The evaluation instruction given to the judge, with the question and the compressed context appended, asks it to check whether the draft is supported by the context, answers exactly what is asked and is short, and to name the correct short answer. v2 adds: never say the answer is unknown, not stated or cannot be determined; if the draft is correct reply exactly "The draft answer is correct."; keep the same short form. v3 adds: only suggest a different answer if the context clearly supports it, and use only words that appear in the context. The full text is in repair.py.

Appendix B: Repository And Reproduction

TABLE XIII
Repository Files

File	Purpose
config.py	Models, sample size, levels, prompt version, cleanup
utils.py	Cached model calls, retries, evaluation loop
metrics.py	EM, F1, answer cleanup, sign/McNemar test
prepare_data.py	Samples 100 HotpotQA questions
compress.py	Plain LLMLingua-2 at 2× to 20×
compress_qa.py	Question-aware compression (paragraphs or sentences)
baseline.py, eval_compressed.py	Uncompressed and compressed evaluation
repair.py	TextGrad repair v1–v3, plain or QA contexts
analyze_repair.py	Helped/hurt, bootstrap ranges, significance
compare_results.py	Paired comparison of two result files
context_recall.py	Answer recall of compressed contexts
cost_analysis.py	Token cost of repair (offline replay)
pipeline.py, demo.py	Middleware and side-by-side demo
make_plots.py, test_metrics.py	Figures; unit tests for scoring

```
python prepare_data.py
python compress.py
python compress_qa.py
python baseline.py
python eval_compressed.py
python eval_compressed.py compressed_qa.json
python compare_results.py results_compressed.json \
    results_qa_promptv1.json
python repair.py v2 ; python repair.py v2 heldout
python repair.py v2 final ratios=5,10
python repair.py v3 final qa ratios=5,10,20
python analyze_repair.py v2 all
python cost_analysis.py        # offline
python demo.py --id 8 --ratio 10
```

References

[1] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. EMNLP*, 2023, arXiv:2310.05736.

[2] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in *Proc. ACL*, 2024, arXiv:2310.06839.

[3] Z. Pan *et al.*, "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Findings of ACL*, 2024, arXiv:2403.12968.

[4] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic 'differentiation' via text," arXiv:2406.07496, 2024.

[5] Y. Li, B. Dong, F. Guerin, and C. Lin, “Compressing context to enhance inference efficiency of large language models,” in *Proc. EMNLP*, 2023, arXiv:2310.06201.

[6] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS Datasets and Benchmarks*, 2023, arXiv:2306.05685.

[7] Z. Yang *et al.*, “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” in *Proc. EMNLP*, 2018.

[8] OpenAI, “gpt-oss-120b and gpt-oss-20b model card,” arXiv:2508.10925, 2025.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, 2024, arXiv:2307.03172.

[10] A. Madaan *et al.*, “Self-Refine: Iterative refinement with self-feedback,” in *Proc. NeurIPS*, 2023, arXiv:2303.17651.

[11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, 2023, arXiv:2303.11366.

[12] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged resources to advance general Chinese embedding,” arXiv:2309.07597, 2023.

[13] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.